AI-GENERATED TEACHING EXEMPLAR - NOT FOR SUBMISSION

DataProof: Verification and Abstention for Trustworthy Tabular Data Analysis

Eraj Zaman | Roll No. 22i1296 | zamaneraj2@gmail.com
Project code: INSERT ASSIGNED CODE | AI4015 Agentic Artificial Intelligence

Abstract— Large-language-model agents can generate plausible tabular analyses while silently misreading a metric, selecting the wrong column, or acting on an underspecified request. This teaching exemplar presents DataProof, a constrained web agent that converts a natural-language question into a machine-readable analysis contract, executes allowlisted pandas operations, independently recomputes the result, and selects among answer, clarification, and abstention. The evaluation uses 54 contract-compatible tasks derived from tables in official DSBench workbooks; these are not the original DSBench questions. Across three runs, a heuristic baseline achieved 62.96% outcome accuracy, whereas DataProof achieved 95.68% with a 1.07 percentage-point standard deviation. Mean task latency was 2.38 s, with 146,402 tokens and 122 execution calls across 162 task runs. In a controlled verifier test, all 28 clean outputs were accepted and all 28 outputs corrupted by a deterministic value change were rejected. Seven main-run failures exposed over-clarification and incorrect row-count semantics. The results support the feasibility of contract-guided execution and calibrated clarification for a narrow class of tabular questions, while the derived benchmark, template-generated questions, small ablation set, and shared execution logic limit generalization.

Keywords— LLM agents, data analysis, verification, abstention, reproducibility, DSBench.

1. Introduction

Language-model agents increasingly combine reasoning with software tools, allowing a user to describe an analytical goal in ordinary language and receive an executable result. This interaction is attractive because it reduces the distance between a question and a data-processing workflow. It also creates a specific reliability problem: a syntactically valid program and a plausible answer do not demonstrate that the agent used the intended metric, filter, grouping, or source. Benchmarks such as DSBench show that realistic data-science work remains difficult for current agents [1]. Related benchmarks broaden the evaluation to notebooks, computer environments, enterprise data, and multi-modal inputs [4]-[9].

DataProof studies a narrower question suitable for a single-semester project: can a structured contract, deterministic execution evidence, and a clarification-or-abstention decision improve the safety of common tabular aggregations? The scope is intentionally limited to CSV/XLSX input and allowlisted operations. This choice makes the agent's intent inspectable and the execution reproducible, while retaining a genuine loop of environment observation, planning, tool use, verification, and action selection.

Three research gaps motivate the design. First, successful execution may still violate the semantics of the request. Second, a review step may judge prose or reproducibility without linking each claim to concrete execution evidence. Third, a capable agent may answer an ambiguous request instead of requesting missing information. These gaps correspond to a JSON contract, an evidence verifier, and a decision controller.

The contribution is empirical rather than architectural novelty. The project provides a reproducible local web application, switchable components for ablation, saved state trajectories, a DSBench-derived data builder with provenance, repeated experiments, and a failure taxonomy. All reported numbers are stored in the repository. The official DSBench aggregate is treated as context only because the task definitions are not identical.

The remainder of this exemplar reviews related research, defines the task and hypotheses, describes the system, reports E1-E6, and discusses failure modes and threats to validity.

2. Related Work

Data-science agent benchmarks. DSBench evaluates agents on realistic analysis and modelling tasks derived from competitions, distinguishing analysis completion from relative modelling performance [1]. Its best reported data-analysis success rate of 34.12% motivates stricter evaluation, but that figure is not a baseline for the derived tasks in this project. DSBC studies context engineering across eight task categories and reports a steep decline as queries combine more categories [3]. DatawiseAgent organizes notebook interaction with a finite-state transducer and makes model-call cost visible [4]. ScienceAgentBench uses executable programs, expert validation, and cost-sensitive evaluation for scientific workflows [5]. DSAEval expands the setting to multiple datasets, modalities, and cumulative queries [6]. DSAgentBench places agents in realistic computer environments with deterministic assessment of analytical outputs [7]. DSGym identifies shortcut-solvable tasks and strengthens data grounding [8]. The Data Agent Benchmark introduces fragmented enterprise sources and realistic data-quality problems [9]. Together these studies indicate that final-answer accuracy alone hides important failures.

Inspection and reproducibility. AIRepr separates an analyst from an inspector and finds a positive relationship between reproducible workflows and analysis accuracy [2]. DataProof adopts that separation but requires a concrete operation contract and runtime evidence. CRITIC similarly uses external tools to critique model outputs [17], while Self-Refine and Reflexion use iterative feedback to revise responses [15], [16]. AutoGen provides a general conversation framework for multiple agents [18]. These systems demonstrate the value of review and feedback, but DataProof restricts review to assertions that can be checked against a table.

Tool-using agents. ReAct interleaves reasoning and action to ground model behavior in an environment [11]. ToolLLM and Toolformer investigate API selection and tool-use learning at scale [14], [23]. PAL and Program of Thoughts separate language reasoning from deterministic computation [24], [25], which directly supports DataProof's decision to let the model specify an operation while pandas performs the arithmetic. Chain-of-thought prompting [26] and code-generation evaluation such as HumanEval [27] provide useful background, although hidden reasoning or executable code alone does not supply semantic proof.

Agent evaluation. AgentBench, GAIA, WebArena, OSWorld, SWE-bench, and MLE-bench evaluate agents across interactive environments, file-backed reasoning, web interaction, operating systems, software maintenance, and machine-learning engineering [12], [13], [19]-[22]. Their common lesson is that credible agent claims require environment interaction, repeatable task definitions, and failure analysis. DataProof applies these principles to a smaller tabular environment.

Selective prediction and abstention. AgentAbstain constructs paired should-act and should-abstain tasks and shows that task-solving ability does not guarantee timely abstention [10]. Selective classification studies the broader risk-coverage trade-off in predictive systems [30]. DataProof adapts this idea by adding controlled ambiguous variants of tabular questions and evaluating whether the system answers or requests clarification appropriately.

Software foundation. The local executor uses pandas, whose dataframe abstraction supports typed columns, missing values, filtering, grouping, and aggregation [28]. Scikit-learn is not required by the current implementation but provides the conventional evaluation context for data-science workflows [29]. The executor's deliberately small operation set improves auditability at the cost of coverage.

The literature supports three design decisions: represent user intent explicitly before execution; retain tool outputs as evidence; and make non-answer actions first-class outcomes. The present work evaluates those decisions rather than claiming a general solution to autonomous data science.

3. Problem Formulation

Let a tabular dataset be D with named columns and a natural-language request be q. The agent must produce one of four externally observable outcomes: an answer y, a clarification request c, an abstention a, or an explicit error e. For answerable questions, the desired answer is the result of an operation o applied to a selected column after zero or more filters and optional grouping. For ambiguous questions, the correct outcome is clarification rather than an invented default.

A contract is represented as the tuple $C = (o, m, F, G, A)$, where m is the metric column, F is an ordered list of filter predicates, G is a list of grouping columns, and A records assumptions. The allowed operations are count, sum, mean, minimum, maximum, median, and number of distinct values. Each filter contains an exact column name, a comparison operator, and a value. The contract is rejected if it references a missing column, an unknown operation, more than four filters, or more than two grouping columns.

Outcome accuracy is one when both the action class and value match the task label. A correct answer requires the status *answered* and an exact executor value; an ambiguous task requires *clarify*. Provider failures are counted separately from model decision errors. Verifier evaluation uses clean outputs and deterministically corrupted copies.

The research questions are:

- **RQ1:** Does the complete contract-guided agent improve outcome accuracy over a limited heuristic baseline on the same derived task set?
- **RQ2:** Does deterministic recomputation reject controlled corruptions while retaining clean outputs?
- **RQ3:** Does an explicit clarification/abstention controller improve appropriate behavior on ambiguous requests?
- **RQ4:** What latency, token, and tool-call costs accompany the complete system?

The corresponding hypotheses predict higher E3 accuracy than E1, high corruption recall without rejecting clean outputs, higher ambiguity accuracy with the controller enabled, and a measurable cost increase from Groq calls and verification.

4. Proposed System

Figure 1 summarizes the pipeline. A profiler observes table shape, names, inferred data types, missing-value counts, and up to three example values per column. Only this profile and the question are sent to the model; the full uploaded dataset stays in the local Python process. The contract planner calls an available Groq chat model with temperature zero and requests a JSON object. A validator enforces the schema before any data operation runs.

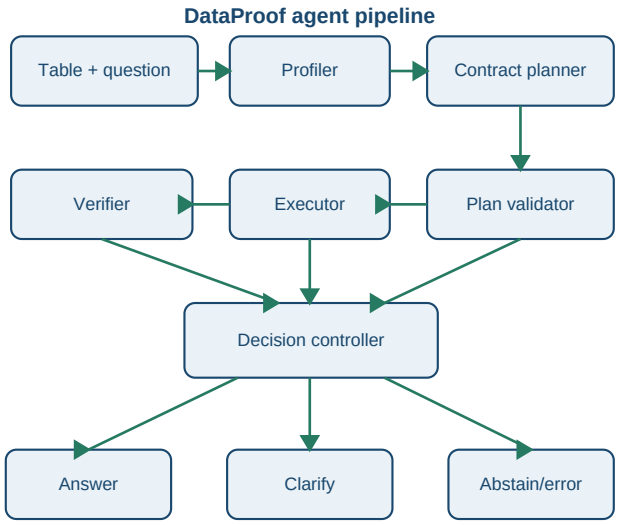

Fig. 1. DataProof separates profiling, contract planning, execution, verification, and action selection.

The executor maps the validated operation to pandas. It coerces numeric filter values only when the source column is numeric and otherwise compares strings. Count without a metric column returns the number of rows, including rows with missing values. Grouped results are returned as records so that group keys remain visible. Arbitrary model-generated Python is never executed.

4.1 Agent Roles, State, and Tools

Although implemented in one process, DataProof has distinct agent roles. The profiler acts as the environment observer. The planner translates language into a proposed contract. The executor is the environment tool. The verifier is an inspector that recomputes the result and produces a verdict. The decision controller selects the user-visible action. This organization gives every gap a switchable component while avoiding an unnecessarily complex message-passing framework.

The trajectory is an ordered list of states with elapsed milliseconds and state-specific payloads. The normal path is profile, plan, contract, execute, verify, and answer. An underspecified plan transitions to clarify. Invalid schemas, unavailable providers, failed verification, and unsupported inputs transition to abstain or error. Each web request receives a random trace identifier and is stored locally as JSON.

The web interface accepts CSV and XLSX files of at most 20 MB, 100,000 rows, and 100 columns. It binds to localhost by default. Duplicate headers and unsupported extensions are rejected. Temporary uploads are removed after processing. API keys are read only from the environment and are excluded by the repository's ignore rules.

4.2 Gap-Specific Components

G1—semantic contract. The prompt defines exact operations and asks the model to use exact column names. The validator ensures that the plan is executable over the observed schema. The component can be ablated by removing the additional contract instruction while retaining local safety checks.

G2—execution evidence. The verifier validates the plan, recomputes the output, checks the number of rows used, and records source columns. It can be disabled for ablation. Because the current verifier calls the same deterministic operation family as the executor, it provides strong protection against value tampering but limited diversity against shared semantic mistakes.

G3—clarification and abstention. The planner can return a concise clarification instead of an operation. The controller preserves that action when enabled. When disabled, an incomplete plan becomes an error, enabling a direct behavioral comparison.

4.3 Robustness and Iterative Improvement

Groq requests use bounded retries for HTTP 429 and transient server errors, exponential waiting capped at 20 seconds, and a recognizable user-agent string. The experiment runner spaces calls to respect the observed token-per-minute quota. Provider failures are labelled as errors instead of being counted as safe abstentions.

The first larger evaluation exposed two application failures. The model sometimes over-clarified an explicit request to total identifier values, and it interpreted a row-count question as a non-null count of one column. The prompt was revised to state the benchmark's explicit semantics, nullable list fields were normalized, and URL errors received retries. On the 18-task validation set containing the observed-failure tables, the revised system scored 18/18. A subsequent reserved-set run was stopped after 37 minutes of network retries and is not treated as a completed experiment.

The development log retains both the failure and its measured effect. This prevents a post-hoc improvement from being presented as if it had been selected before observing the validation behavior.

4.4 Reproducibility

The repository contains the builder, manifests, provenance, prompts, dependency file, environment example, unit tests, experiment scripts, saved task-level results, analysis script, architecture, representative traces, and a tagged final code version. Six deterministic tests cover filtering, aggregation, schema rejection, output corruption, row counting, nullable model fields, and clarification behavior.

5. Experimental Setup

Data source. The processed DSBench workbook archive contains competition spreadsheets and question files. A deterministic builder scans workbook sheets, detects rectangular tables with at least two numeric columns, retains at most 5,000 rows, and records workbook, sheet, header row, column names, and competition identifier. Generated CSV derivatives are excluded from Git because the DSBench code license does not cover external datasets.

Derived tasks. For each detected table, the builder creates questions for mean, sum, maximum, minimum, median, row count, filtered sum, an ambiguous metric, and a missing comparison. Expected values and oracle plans are calculated locally. The later refined split contains 9 development tasks, 18 validation tasks, and 36 reserved tasks. The main completed run used the original fixed 54-task evaluation set from six tables. Because the questions are generated from templates, the experiment measures contract handling over real workbook values rather than performance on the official DSBench questions.

Configurations. E1 uses the offline heuristic planner with the contract prompt, verifier, and abstention controller disabled. E2 records the 34.12% aggregate reported by DSBench as contextual evidence and explicitly marks it as not reproduced. E3 uses the complete system with Groq's *openai/gpt-oss-20b*, temperature zero, and the same local executor. E4 tests each gap: complete versus baseline behavior, deterministic verifier fault injection, and ambiguity categories. E5 disables each component on the nine-task pilot. E6 summarizes latency, tokens, tool calls, and provider errors.

Repeats and hardware. E1 and E3 run every main task three times. The computer has an Intel Core i7-10510U processor, 16 GB RAM, and integrated Intel UHD graphics. Model inference is hosted by Groq, so the local GPU does not constrain inference. The local machine loads workbooks, executes pandas operations, verifies outputs, and stores traces.

6. Evaluation Methodology

Outcome accuracy is computed per run and then averaged across repeat-level accuracies. Standard deviation is the sample standard deviation of the three repeat accuracies. The main result reports percentage points to avoid implying more precision than the evaluation supports.

For answerable tasks, exact equality is used because both expected and predicted values are produced by deterministic pandas operations over the same saved table. For ambiguity tasks, correctness depends on the action class. A clarification is correct only when the manifest labels the request as underspecified.

The verifier experiment includes each answerable reserved task twice: once with the clean executor output and once after adding one to a scalar value. Clean acceptance is the fraction of unmodified outputs that pass. Corruption recall is the fraction of modified outputs that fail. This design tests deterministic tampering sensitivity and does not approximate the full distribution of model mistakes.

Failure analysis assigns incorrect main-run outcomes to over-clarification, wrong execution semantics, service/schema error, or another decision error. The repository stores all failed trajectories and two representative successes and failures. Efficiency uses end-to-end runner latency, API token usage returned by Groq, and the number of executor tool calls.

The published DSBench percentage is not placed in a shared metric column with DataProof because the official and derived tasks differ. Any visual comparison would be misleading; it is cited only as evidence that realistic official tasks are difficult.

7. Results

Configuration	Tasks × repeats	Accuracy	SD	Mean latency
E1 heuristic	54 × 3	62.96%	0.00 pp	12 ms
E3 DataProof	54 × 3	95.68%	1.07 pp	2,375 ms

Table I. Main completed evaluation. Standard deviation is across three repeat-level accuracies.

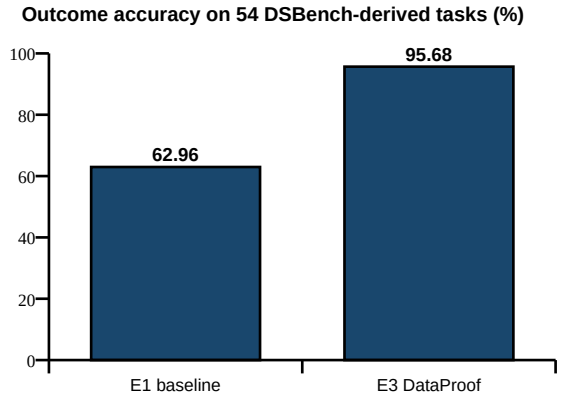

Fig. 2. Descriptive outcome accuracy on the shared derived evaluation set.

DataProof improved mean outcome accuracy by 32.72 percentage points over the heuristic baseline on the shared derived task set. E3 repeat variability was 1.07 percentage points, while the deterministic heuristic produced identical repeat scores. The complete run contained no provider errors.

By category, ambiguity and missing-comparison questions, minimum, maximum, mean, and median achieved 100% accuracy across 18 runs per category. Sum achieved 94.44%. Count and filtered sum each achieved 83.33%. These category results identify the semantics of identifiers and missing values as the main remaining weaknesses.

7.1 Gap Resolution

Gap	Evidence	Assessment
G1 contract	E3 95.68%; no-contract pilot 88.89%	Partially addressed
G2 verifier	28/28 clean accepted; 28/28 corruptions rejected	Partially addressed
G3 abstention	100% ambiguity categories; no-abstention pilot 66.67% overall	Partially addressed

Table II. Gap-resolution summary. 'Partially' reflects scope and evaluation limitations.

G1 receives partial support because the complete system outperformed the heuristic and the smaller no-contract pilot scored lower than the full pilot. The comparison does not isolate the contract on the 54-task set, so a general causal claim is not justified. G2 receives partial support because the verifier perfectly detected the tested scalar corruptions, but the no-verifier pilot tied the complete pilot. G3 receives partial support because ambiguous main-run categories were handled correctly and disabling the controller reduced pilot outcomes.

7.2 Base-Paper Context

DSBench reports 34.12% success for its best evaluated agent on official data-analysis tasks [1]. DataProof's 95.68% must not be interpreted as outperforming that result: the derived questions cover a much smaller operation space and are evaluated with different task definitions. The base-paper number is therefore reported as context, not entered into a common comparative test.

7.3 Verifier Fault Injection

The reserved verifier experiment contained 28 answerable tasks. Every clean output was accepted and every output modified by adding one was rejected. This produces 100% clean acceptance and 100% corruption recall for the tested transformation. The experiment verifies the implementation and demonstrates evidence sensitivity, but it does not cover wrong-column plans or mutually consistent executor-verifier errors.

8. Ablation and Comparative Analysis

Pilot configuration	Accuracy	Tokens	Latency
Full DataProof	100.00%	12,756	1,405 ms
No contract instruction	88.89%	12,561	757 ms
No verifier	100.00%	12,624	2,497 ms
No abstention control	66.67%	12,709	1,939 ms

Table III. Directional ablations on the nine-task synthetic pilot, three repeats.

Removing the contract instruction reduced pilot accuracy by 11.11 percentage points. Inspection showed that the unconstrained planner answered one intentionally vague question instead of clarifying. Removing the abstention controller reduced accuracy by 33.33 percentage points because the three ambiguous questions became errors. Removing the verifier did not change pilot accuracy, demonstrating that the pilot had no natural wrong execution for the verifier to catch.

The ablation set is too small for a strong quantitative conclusion. Its purpose is to confirm that the switches work and expose the behavior associated with each component. The 54-task E3 comparison and the 28-task fault-injection experiment provide stronger evidence for end-to-end performance and verifier sensitivity, respectively.

8.1 Efficiency and Reliability

E3 used 146,402 total API tokens and 122 executor calls over 162 task runs. Mean end-to-end task latency was 2,375 ms. The heuristic baseline required no API tokens and averaged 12 ms, illustrating the cost of language interpretation. The complete run recorded zero provider errors after pacing and bounded retries.

Tool-call count is lower than total task runs because clarification tasks do not execute a data operation. This is desirable: the decision controller avoids unnecessary work when intent is incomplete. Token use is dominated by the schema profile and JSON response. Future work could reduce profile size through column retrieval or a compact schema representation.

9. Failure Analysis

Seven of 162 E3 runs were incorrect. Four were over-clarifications. In one table, 'Invoice Number' is numeric but semantically resembles an identifier; the model sometimes asked whether 'total' meant sum or count despite the explicit phrasing. Three failures were wrong execution semantics. For a table with missing values in 'List of Teams', the planner selected that column for a row count, returning 20 non-null values rather than all 23 rows.

Representative success 1. For a filtered-sum question, the planner selected sum, used the exact target column, created a numeric greater-than filter, executed pandas, and passed independent recomputation. **Representative success 2.** For 'What changed in this dataset?', the system requested the missing comparison context and made no tool call.

Representative failure 1. The agent over-clarified an explicit request to total invoice numbers. This is conservative but incorrect under the benchmark contract. **Representative failure 2.** The agent answered a row-count question by counting non-null values in one column. Verification passed because the plan and output were internally consistent, exposing a shared semantic failure rather than arithmetic corruption.

These cases show why independent recomputation is necessary but insufficient. A verifier that accepts the same plan cannot detect a semantically wrong plan. Stronger verification should compare the question and contract, use separately prompted plan critics, or test counterfactual plans before acceptance.

After the failures, row-count semantics were stated explicitly and nullable list fields were normalized. The 18-task validation rerun reached 100%. Because the reserved run was interrupted by network retries, the improvement is reported as validation evidence rather than a final generalization result.

10. Limitations and Threats to Validity

Construct validity. The derived questions test common aggregations, filters, and ambiguity decisions. They do not represent the modelling, visualization, spreadsheet formula construction, or multi-file reasoning in official DSBench. Consequently, outcome accuracy measures compliance with this constrained contract rather than general data-science expertise.

Internal validity. Expected answers are generated by pandas using oracle plans, and the executor uses the same library. This reduces labelling error but may hide shared implementation assumptions. The verifier reuses the deterministic operation family, so its perfect corruption result should not be generalized to wrong-column or wrong-intent failures.

External validity. Tables come from seven detected workbook sheets, with at most 5,000 rows per table. The table detector favors rectangular numeric data and excludes less regular sheets. The results may not transfer to nested spreadsheets, multiple worksheets, text-heavy data, dates requiring specialized semantics, or high-dimensional datasets.

Statistical conclusion validity. Repeats use temperature zero, although hosted inference can still vary. The three repeat scores provide a descriptive standard deviation but not a confidence interval over a well-defined population. The ablation has nine tasks and must be interpreted as directional. No hypothesis test is reported.

Service dependence. Groq model availability and quotas can change. The project observed browser-signature blocking, model retirement, token-rate limits, DNS resets, and a prolonged retry episode. The repository records the selected model and handles bounded failures, but long-term reproducibility requires pinning an accessible model or saving compatible local weights.

Privacy and security. The full table remains local, but the profile includes column names, missing counts, and example values that may still be sensitive. Production deployment would need data classification, redaction, authentication, encrypted storage, and a sandboxed server. The current localhost web interface is a research prototype.

Academic and documentation limits. This document is an AI-generated teaching exemplar and is not eligible as a student-authored submission. Citation metadata must be independently verified. Official Turnitin reports can only be generated by the institution from the final student-written paper.

11. Conclusion

DataProof demonstrates a compact agentic workflow for auditable tabular analysis. A model observes a schema profile and proposes a structured contract; a constrained local tool executes the contract; a verifier recomputes the result; and a controller answers, clarifies, or abstains. On 54 DSBench-derived tasks repeated three times, the complete system achieved 95.68% mean outcome accuracy compared with 62.96% for a heuristic baseline. A controlled verifier experiment accepted all clean outputs and rejected all injected scalar corruptions.

The failure analysis is as important as the headline score. Over-clarification shows the trade-off between caution and task completion, while the row-count failures show that internally consistent evidence cannot repair a semantically wrong plan. Future work should add an independent question-to-contract critic, wider operations, multi-sheet support, a completed reserved-set rerun, larger ablations, calibrated risk-coverage analysis, and evaluation on official benchmark questions where licensing and implementation permit.

The main practical contribution is a reproducible evidence trail: each result has a manifest entry, state trajectory, configuration, task-level outcome, latency, token count, and tool-call record. This makes negative and partial results visible and supplies a foundation for more rigorous evaluation.

References

[1] L. Jing et al., “DSBench: How Far Are Data Science Agents from Becoming Data Science Experts?,” ICLR, 2025.

[2] Q. Zeng et al., “AIRepr: An Analyst-Inspector Framework for Evaluating Reproducibility of LLMs in Data Science,” Findings of EMNLP, 2025, doi:10.18653/v1/2025.findings-emnlp.539.

[3] R. M. R. Kadiyala et al., “DSBC: Data Science Task Benchmarking with Context Engineering,” IJCNLP-AACL, 2025.

[4] Z. You et al., “DatawiseAgent: A Notebook-Centric LLM Agent Framework for Adaptive and Robust Data Science Automation,” EMNLP, 2025, doi:10.18653/v1/2025.emnlp-main.58.

[5] Z. Chen et al., “ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery,” ICLR, 2025.

[6] M. Sun et al., “DSAEval: Evaluating Data Science Agents on a Wide Range of Real-World Data Science Problems,” EMNLP, 2026, doi:10.18653/v1/2026.emnlp-main.970.

[7] M. Rahman et al., “DSAgentBench: Can Agents Automate End-to-End Data-Science Workflows in Real Computer Environments?,” EMNLP, 2026, doi:10.18653/v1/2026.emnlp-main.107.

[8] F. Nie et al., “DSGym: A Holistic Framework for Evaluating and Training Data Science Agents,” ICML, 2026.

[9] R. Ma et al., “Can AI Agents Answer Your Data Questions? A Benchmark for Data Agents,” EMNLP, 2026, doi:10.18653/v1/2026.emnlp-main.1872.

[10] X. Liu et al., “AgentAbstain: Do LLM Agents Know When Not to Act?,” NeurIPS, 2026.

[11] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” ICLR, 2023.

[12] X. Liu et al., “AgentBench: Evaluating LLMs as Agents,” ICLR, 2024.

[13] G. Mialon et al., “GAIA: A Benchmark for General AI Assistants,” ICLR, 2024.

[14] Y. Qin et al., “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs,” ICLR, 2024.

[15] N. Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” NeurIPS, 2023.

[16] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” NeurIPS, 2023.

[17] Z. Gou et al., “CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing,” ICLR, 2024.

[18] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” arXiv:2308.08155, 2023.

[19] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” ICLR, 2024.

[20] J. S. Chan et al., “MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering,” ICLR, 2025.

[21] T. Xie et al., “OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments,” NeurIPS, 2024.

[22] S. Zhou et al., “WebArena: A Realistic Web Environment for Building Autonomous Agents,” ICLR, 2024.

[23] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” NeurIPS, 2023.

[24] L. Gao et al., “PAL: Program-aided Language Models,” ICML, 2023.

[25] W. Chen et al., “Program of Thoughts Prompting: Disentangling Computation from Reasoning,” TMLR, 2023.

[26] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS, 2022.

[27] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.

[28] W. McKinney, “Data Structures for Statistical Computing in Python,” Proc. SciPy, 2010, doi:10.25080/Majora-92bf1922-00a.

[29] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” JMLR, vol. 12, 2011.

[30] Y. Geifman and R. El-Yaniv, “Selective Classification for Deep Neural Networks,” NeurIPS, 2017.